



Monolith to Microservices

Why, When, and How to Migrate



Agenda

1. Introduction to Monoliths and Microservices
2. Deciding When to Migrate
3. Migration Strategies and Patterns
4. Technical Challenges in Migration
5. Operational Considerations
6. Best Practices and Final Thoughts



Introduction to Monoliths and Microservices



Monolith to Microservices

Monolithic to Microservices Evolution

Transition from unified codebase to independent services

Motivations for Migration

Scalability, team autonomy, and deployment speed

Challenges and Strategies

Addressing data ownership, gradual migration, and new risks



What Is a Monolith?

Unified Codebase and Deployment

All features and logic reside in a single project.

Shared Database Structure

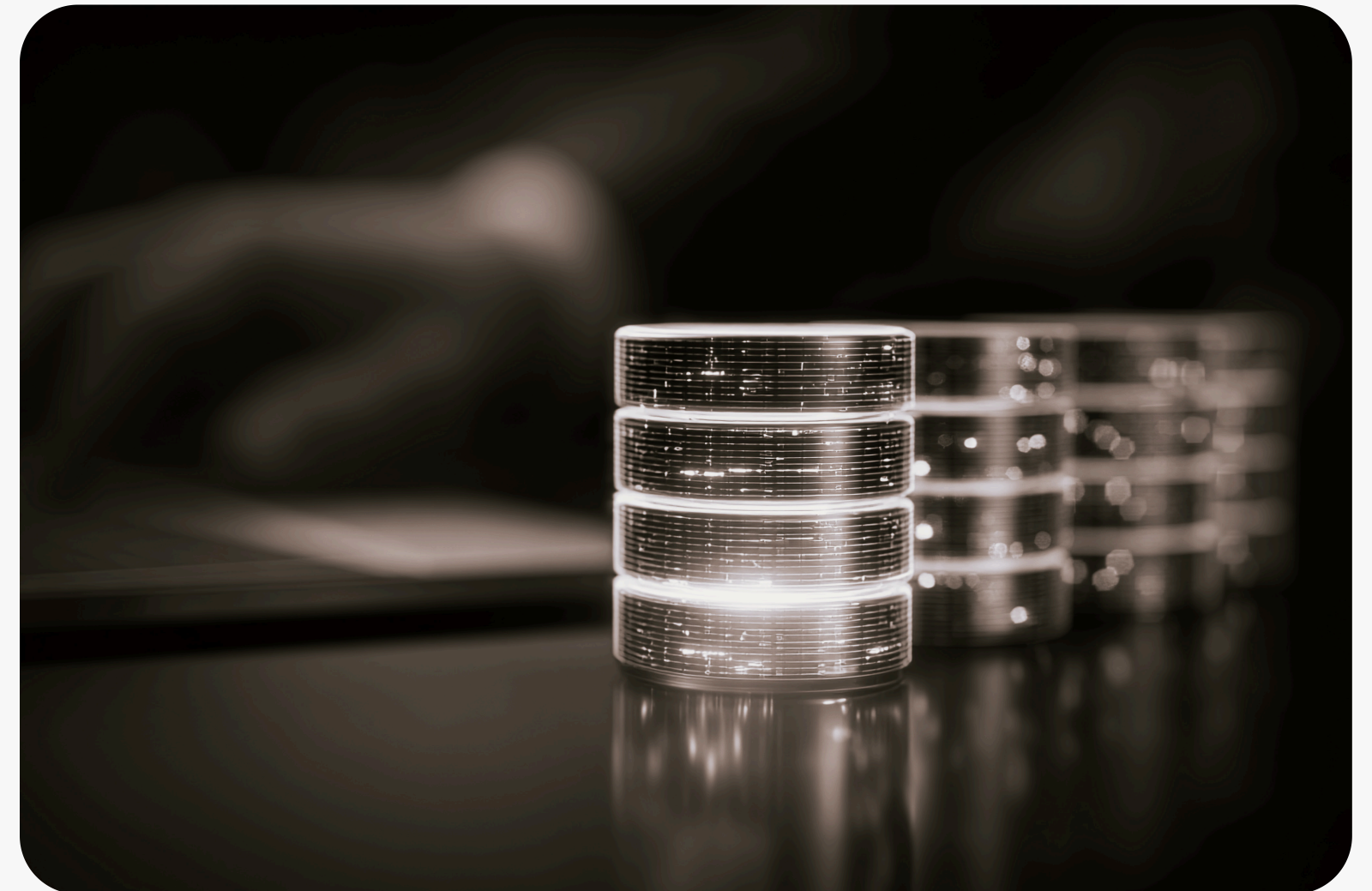
Modules access the same database, increasing coupling.

Tightly Coupled Modules

Changes in one area can impact the entire system.

Logical Separation, Unified Runtime

Code may be organized, but runs as one application.





Why Monoliths Are Popular

Rapid Product Launch

Monoliths enable quick initial releases.

Streamlined Development

Local setup and coding are straightforward.

Effortless Testing & Debugging

Unified codebase simplifies troubleshooting.

Fast Delivery Cycles

Teams can ship features rapidly.





Problems with Large Monoliths

Deployment Bottlenecks

Entire system must redeploy for any change.

Tight Module Coupling

Code dependencies hinder parallel development and collaboration.

Scaling Inefficiencies

Resources scale for all features, not just those in demand.

High Blast Radius

Small bugs can impact the whole application.



What Are Microservices?

Independent, Focused Services

Each microservice handles a specific business function.

Own Data and Database

Services manage their own data, avoiding shared databases.

Autonomous Deployment & Scaling

Services can be deployed and scaled without affecting others.

API or Event-Based Communication

Services interact via APIs or asynchronous events.



Deciding When to Migrate



When Should You Move?

Scaling for Large Teams

Microservices enable parallel development and ownership.

Independent Feature Evolution

Services can be updated and deployed separately.

Frequent and Safe Deployments

Smaller services reduce risk and speed up releases.

Clear Domain Boundaries

Well-defined contexts simplify migration and maintenance.



When NOT to Use Microservices

Best for Small Teams and Startups

Monoliths simplify development and reduce overhead.

Ideal for Simple CRUD Applications

Microservices add unnecessary complexity for basic systems.

Requires DevOps Maturity

Microservices demand automation and strong operational practices.

Monoliths Excel in Simplicity

Unified codebase makes testing and deployment straightforward.



Monolith vs Microservices Visual



Monolith: Unified System

Single codebase, logic, and shared database



Microservices: Isolated Services

Each service owns its code, logic, and data



Separation and Independence

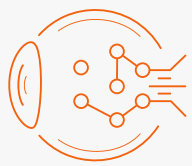
Services communicate via APIs or events



Migration Strategies and Patterns



Biggest Migration Mistake



Avoid All-at-Once Rewrites

Attempting a full system rewrite disrupts stability.



High Risk and Cost

Big Bang migrations often exceed budgets and timelines.

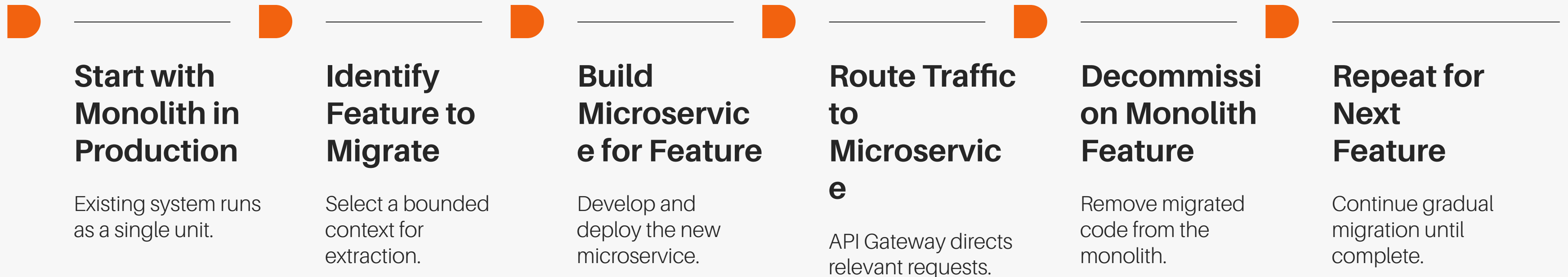


Frequent Failure Rate

Many large-scale rewrites never reach production.



Strangler Fig Pattern





Migration Flow Step-by-Step

Define Clear Business Boundaries

Identify distinct domains or modules within your monolith.

Extract and Isolate a Service

Move one bounded context into a standalone microservice.

Implement an API Gateway

Route requests to either the monolith or new microservice as needed.

Redirect Traffic and Retire Old Code

Gradually shift traffic and decommission replaced monolith components.



Technical Challenges in Migration



Database Migration Challenge



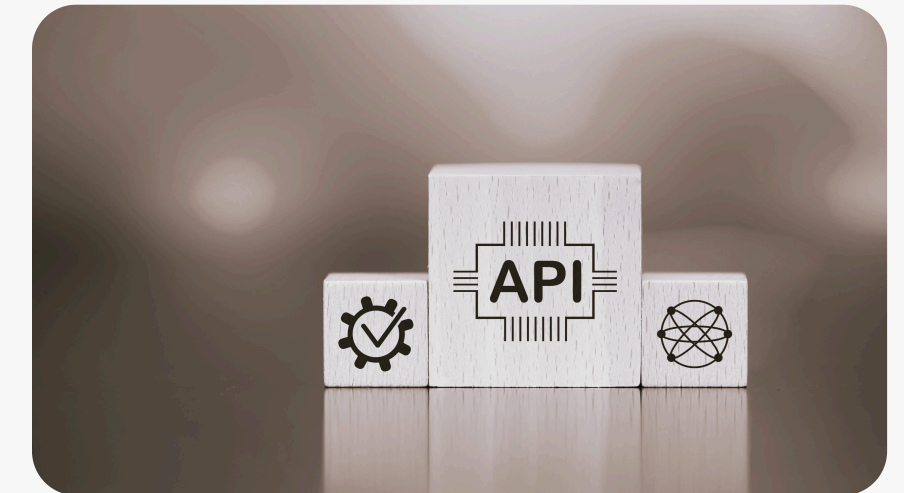
Shared Database Creates Tight Coupling

Migrating away from a single database is complex.



Microservices Must Own Their Data

Each service manages its own schema and storage.



Data Sharing via APIs or Events

Services exchange data through APIs or event streams.



Handling Transactions



Monolith Transaction Model

Single database enables ACID transactions.



Distributed Transactions Challenge

Microservices avoid cross-service transactions.



Saga Pattern Introduction

Orchestrates multi-step business processes via events.



Eventual Consistency

System state converges over time, not instantly.



Authentication & Security



Centralized Authentication Service

Single service manages user identity and login.



Secure Service-to-Service Communication

Use JWT tokens or API keys for authentication between services.



Authorization at Service Boundaries

Each microservice enforces access control for its own resources.



Communication During Migration



Begin with RESTful APIs

Use simple HTTP endpoints for initial service communication.



Transition to Event-Driven Messaging

Adopt events for scalable, asynchronous interactions as system grows.



Minimize Chatty Interactions

Design APIs to reduce excessive back-and-forth between services.



Operational Considerations



Observability Is Mandatory

Centralized Logging

Aggregate logs from all services for unified visibility.

Metrics Collection

Track performance and health across microservices.

Distributed Tracing

Trace requests end-to-end to pinpoint issues quickly.

Operational Clarity

Essential for debugging and managing complex systems.



Deployment Changes

Deploy Services Independently

Each microservice is released on its own schedule.

Automate with CI/CD Pipelines

Continuous integration and delivery streamline releases.

Leverage Containers for Consistency

Docker and similar tools ensure reliable environments.

Manual Deployments Don't Scale

Automation is essential as service count grows.



New Problems Introduced

Network Reliability Risks

Service calls can fail due to network issues.

Data Consistency Challenges

Maintaining consistency across services is complex.

Debugging Distributed Systems

Tracing bugs across services requires advanced tools.

Operational Overhead

Managing many services increases maintenance effort.





Best Practices and Final Thoughts



Best Practices for Migration

Begin with Small, Manageable Steps

Extract one service at a time to reduce risk.

Automate Processes Early

Implement CI/CD pipelines for reliable deployments.

Define Clear Domain Boundaries

Ensure each microservice owns its logic and data.

Invest in DevOps Capabilities

Build strong infrastructure for monitoring and operations.





Final Takeaways

Start with a Monolith for Simplicity

Monoliths are ideal for early-stage products and teams.

Migrate Gradually for Safety

Incremental migration reduces risk and keeps systems stable.

Microservices Are a Strategic Trade-Off

Adopt microservices only when scaling and independence are needed.

Align Architecture with Team Structure

Design services to match how your teams work and collaborate.